Scheduling

Introduction to Scheduling Process Behavior

CPU bound vs. I/O bound processes

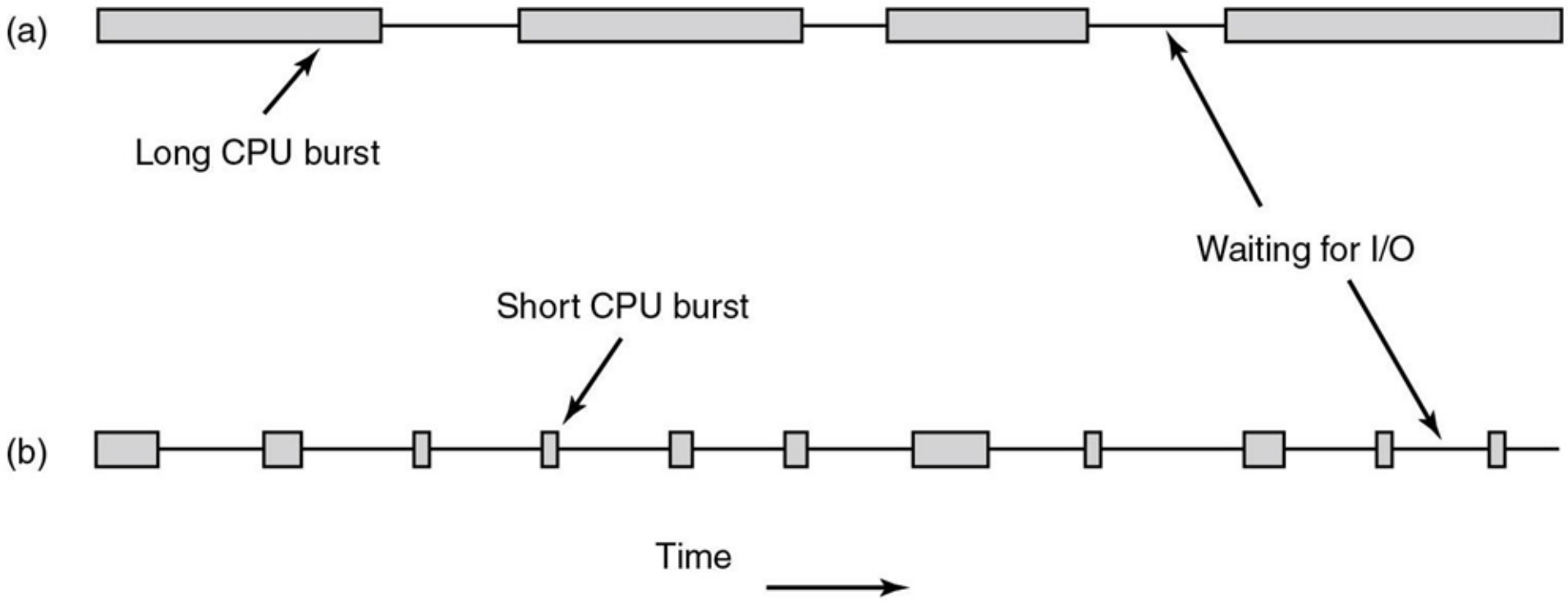
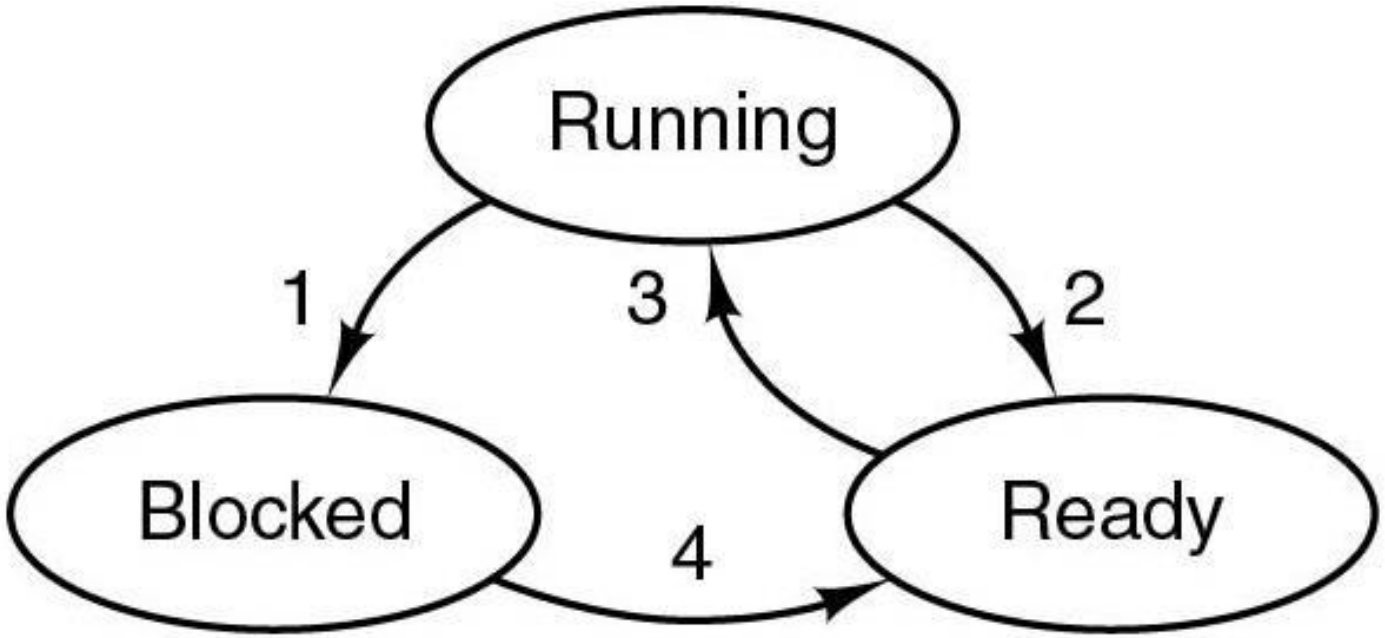


Figure 2.40 Bursts of CPU usage alternate with periods of waiting for I/O. (a) A CPU-bound process. (b) An I/O-bound process.

Process State Revisited



If more processes ready than CPU s available:

- **Scheduler** decides which process to run next
- Algorithm used by scheduler is called **scheduling algorithm**

When to Schedule?

- ***Process exits***
- ***Process blocks on I/O, Semaphore, etc.***
- *When a new process is created*
- *When an interrupt occurs:*
  - *I/O, clock, syscall, etc.*

Preemptive vs **non-preemptive** scheduling?

Categories of Scheduling Algorithms

- Batch System - non-preemptive scheduling
- Interactive System - preemptive scheduling
- Real Time System - different category

Scheduling Algorithm Goals (1 of 2)

- Different goals for different systems
  - Batch
  - Interactive
  - Real time
- All systems:
  - Fairness - giving each process a fair share of the CPU
  - Policy enforcement - seeing that stated policy is carried out
  - Balance - keeping all parts of the system busy

Batch Systems

Scanrail/123RF

- Throughput : maximize jobs per hour
- Turnaround time : minimize time between submission and termination
- CPU utilization : keeping the CPU busy all the time

First-Come First-Served

- Process jobs in order of their arrival
- Non-preemptive
- Single Process Queue
  - New jobs or blocking processes are added to the end of the queue
- “Convoy Effect” if only few CPU bound and many I/O bound processes

Shortest Job First (1 of 2)

- Pick the job with the shortest run time
- Provably optimal:
  - Lowest turnaround time

Shortest Job First (2 of 2)

- Pick the job with the shortest run time
- Provably optimal:
  - Lowest turnaround time
  - Only if all jobs are available simultaneously
  - If new jobs arrive, it may lead to starvation
- Runtimes have to be known in advance
- Highest-response-ratio-next
  - Improved version of Shortest Job First

Interactive Systems

- **Response time:** respond to requests quickly
- **Proportionality:** meet users' expectations

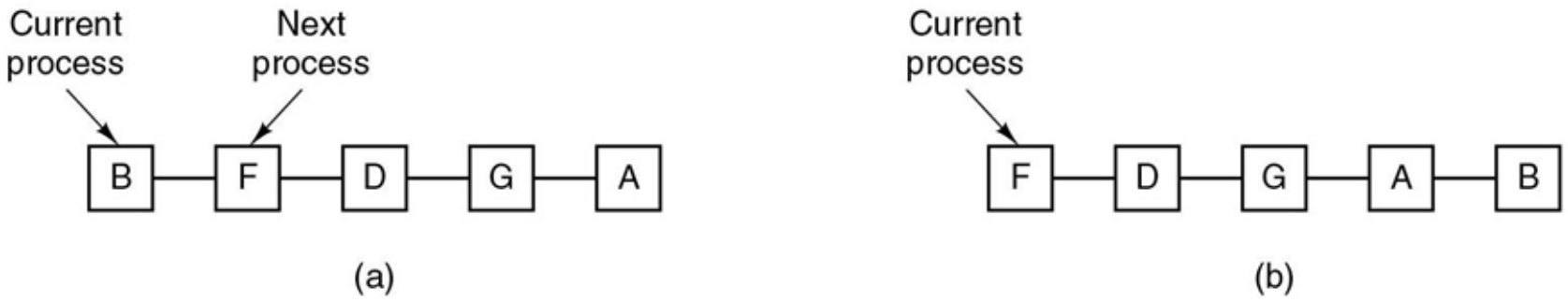
McClatchy-Tribune/Tribune Content Agency L L C/Alamy Stock Photo

Apple MacIntosh (1984)

Round Robin Scheduling

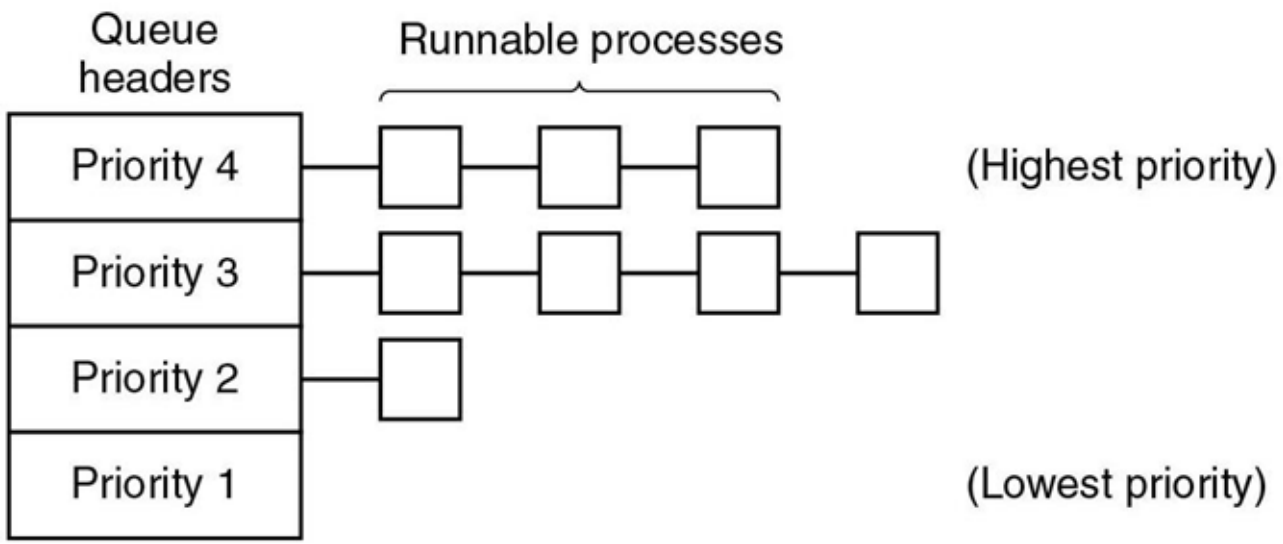
- Preemptive scheduling algorithm
- Each process gets a **time slice** or **quantum**
- If process is still running at end of quantum it gets preempted end goes to end of ready queue
- Question: How big should the quantum be?

CPU utilization vs. response time



Priority Scheduling (1 of 2)

Simplest, multiple queues



- Similar to round robin but several ready queues
- Next process is picked from queue with highest priority
- Static vs. dynamic priorities
- What processes should have high priorities?

Priorities Are Complicated

and may not work out the way you think...

(remember priority inversion?)

Shortest Process Next

Problem: How to minimize response time for each priority queue?

Idea: Use shortest "job" first and try to best predict next running time

Solution: Form weighted average of previous running times of process $\rightarrow$ **Aging**

$$T_{k+1} = a * T_{k-1} + (1 - a) * T_k$$

Easy to implement when $a = 1/2$

Guaranteed Scheduling

Idea: N processes running $\longrightarrow$ each process gets 1/Nth of CPU time
(also known as fair-share)

Solution:

- Calculate how much C P U time it might have gotten:
  - Time since process creation divided by N
- Measure actual consumed C P U time and form ratio
- 0.5 $\longrightarrow$ process running half the time it was entitled to
- 2.0 $\longrightarrow$ process running twice as much as it was entitled to
- Pick process with the smallest ratio to run next
- Note: fair-share scheduling can be per-user/-process

Lottery Scheduling

- Processes get lottery tickets
- Whenever a scheduling decision is made O S chooses a winning ticket randomly

- Processes can possess multiple tickets $\longrightarrow$ Priorities

- Tickets can be traded between processes.
- Tickets are immediately available to newly created processes.

Policy vs. Mechanism

Important principle

Here: we may have a scheduling algorithm, but parameters to be filled in by user (process)

For instance, to give some child processes higher priority than others

Real Time Systems (1 of 3)

Yauhen_D/Shutterstock

N A S A

- Meeting deadlines: avoid losing data
- Predictability: avoid quality degradation in multimedia systems

Real Time Systems (2 of 3)

- Systems where timing plays essential role
- Soft real time vs. Hard real time
- Can consist of periodic and aperiodic tasks
- Schedules can be static or dynamic
- System with periodic tasks is schedulable when:

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

Real Time Systems (3 of 3)

Consider three jobs:

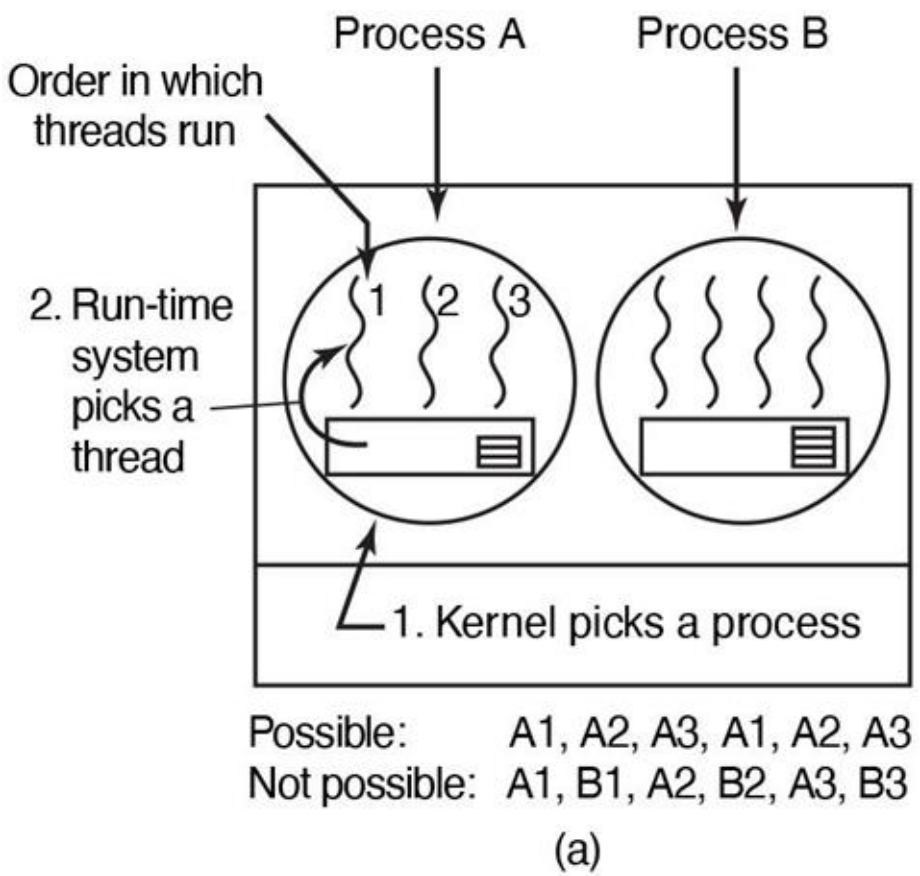
-	**Period**	**Req. C P U time**
P1	100 ms	50 ms
P2	200 ms	30 ms
P3	500 ms	100 ms

Is this system schedulable?

Scheduling Algorithm Goals (2 of 2)

- Interactive systems
  - Response time - respond to requests quickly
  - Proportionality - meet users' expectations
- Real-time systems
  - Meeting deadlines - avoid losing data
  - Predictability - avoid quality degradation in multimedia systems

Thread Scheduling (1)



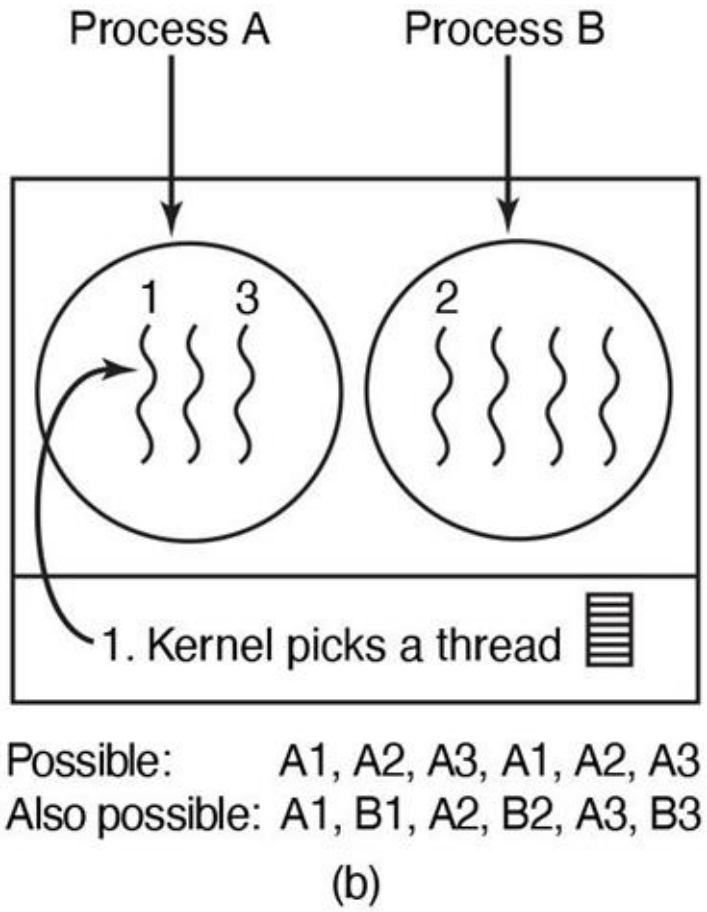


Figure 2.45. (a) Possible scheduling of user-level threads with a 50-msec process quantum and threads that run 5 msec per CPU burst.

Figure 2.45 (b) Possible scheduling of kernel-level threads with the same characteristics as (a).

Copyright

This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.